CHƯƠNG 2 · BÀI 2

2.2. useContext & useReducer

Tài liệu học tập ReactJS Chuyên Sâu · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Dùng Context để tránh prop drilling
> - Dùng Hook useReducer quản lý state phức tạp
> - Kết hợp Context với Reducer để quản lý state toàn ứng dụng

1. Prop drilling và Context

Context là cơ chế truyền dữ liệu xuống nhiều tầng component mà không cần truyền props qua từng tầng. Tình trạng phải truyền props qua nhiều tầng trung gian được gọi là prop drilling. Context được tạo bằng `createContext`, cung cấp bằng Provider và đọc bằng Hook `useContext`.

```
const ThemeContext = createContext("sang");

function App() {
  return (
    <ThemeContext.Provider value="toi">
      <NutBam />
    </ThemeContext.Provider>
  );
}

function NutBam() {
  const theme = useContext(ThemeContext);
  return <button className={theme}>Bấm</button>;
}
```

2. useReducer

`useReducer` là Hook quản lý state phức tạp bằng một hàm reducer. Reducer là hàm thuần nhận state hiện tại và action, rồi trả về state mới. Component gửi action bằng hàm `dispatch`, ví dụ `dispatch({ type: 'add', payload: item })`.

```
function reducer(state, action) {
  switch (action.type) {
    case "add":    return [...state, action.payload];
    case "remove": return state.filter((x) => x.id !== action.id);
    default:       return state;
  }
}
const [gioHang, dispatch] = useReducer(reducer, []);
```

3. Khi nào dùng

Dùng `useState` cho state đơn giản. Dùng `useReducer` khi state có nhiều trường hoặc nhiều cách cập nhật liên quan nhau. Dùng Context khi nhiều component ở các tầng khác nhau cần cùng dữ liệu như người dùng đăng nhập, giao diện sáng hoặc tối.

4. Lưu ý hiệu năng

Mỗi khi giá trị của Provider thay đổi, mọi component dùng context đó đều render lại. Nên tách context theo chức năng (đăng nhập, giao diện) thay vì gom mọi thứ vào một context lớn.

Tóm tắt

- Context tránh prop drilling: `createContext`, Provider, `useContext`.
- `useReducer` quản lý state phức tạp bằng reducer thuần và `dispatch` action.
- Tách context theo chức năng để hạn chế render lại không cần thiết.

Câu hỏi ôn tập và bài tập vận dụng

1. Tạo ThemeContext cho phép đổi giao diện sáng và tối toàn ứng dụng.
2. Viết giỏ hàng bằng useReducer với các action thêm, xoá, đổi số lượng.
3. Giải thích khi nào nên chọn useReducer thay cho useState.

useContext & useReducer — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhiều trang cần user hiện tại hoặc giỏ thao tác. Context truyền giá trị, reducer mô tả cách chuyển trạng thái; không thay thế mọi state cục bộ bằng context toàn ứng dụng.

```
function reducer(state, action) {
  switch (action.type) {
    case "reset": return {items: []};
    case "add": return {...state, items: [...state.items, action.item]};
    default: return state;
  }
}
```

Tự luyện: thêm rồi reset, kiểm tra state đầu không bị sửa. **Hướng giải:** reducer không gửi HTTP hoặc tạo side effect; auth context không phải lớp bảo mật thay cho API. Backend vẫn kiểm tra token và quyền ở mọi request.

Nguồn học tiếp

https://react.dev/learn